To get the performance advantages you'll want to regularly refresh your materialized views, which can be done both from the application code or with SQL.

The FastSearchResult model has a refresh method. From bin/rails console, refresh the view like this:

```
 FastSearchResult.refresh
   (20.7ms)  REFRESH MATERIALIZED VIEW "fast_search_results";
=> #<PG::Result:0x0000000115bc40d8 status=PGRES_COMMAND_OK \
    ntuples=0 nfields=0 cmd_tuples=0>
```

Compare the response times between SearchResult.first and FastSearchResult.first. The materialized view will return much faster!

Rideshare keeps the materialized view results refreshed using a cron job. Ruby on Rails does not offer native cron job scheduling support so the third party gem whenever[21] is used to generate and manage jobs.

Review config/schedule.rb. This code uses the Rails runner[22] command to invoke the same code you called earlier from bin/rails console.

```
every 15.minutes do
  runner "FastSearchResult.refresh"
end
```

Materialized Views can be refreshed concurrently as well, although they do require a Unique index to be added. Rideshare added a Unique index to fast_search_results to enable concurrent refreshes.

To refresh concurrently from psql, run this statement:

```
REFRESH MATERIALIZED VIEW CONCURRENTLY "fast_search_results";
```

You've now seen how to leverage database views with high performance materialized views, that can be indexed and refreshed concurrently.

Besides these techniques, Ruby on Rails adds caching layers for database queries that help improve your application performance. Let's see what types of caches are available and how to leverage them.

## Reducing Queries With Active Record Caches

Active Record has a variety of caches that run to improve the performance of your Rails application code.

21. https://github.com/javan/whenever
22. https://guides.rubyonrails.org/command_line.html#bin-rails-runner